

AI 编程最佳实践指南

从入门到精通 · 2026 年最新版

编写日期：2026 年 6 月

适用工具：Cursor · Trae · Windsurf · Copilot · Claude Code

技术栈：不限（通用方法论）

目录

- 一、AI 编程思维重塑
- 二、提示词工程核心技巧
- 三、开发环境配置最佳实践
- 四、项目级 AI 协作工作流
- 五、代码质量与安全
- 六、常见陷阱与避坑指南
- 七、进阶：Agent 与 MCP
- 八、工具选型与对比
- 九、AI 编程 Checklist

一、AI 编程思维重塑

1.1 核心认知转变

传统编程	AI 辅助编程
我是程序员，AI 是工具	我是架构师，AI 是高级初级工程师
一行一行写代码	设计需求→AI 生成→我审查→迭代优化
遇到问题查 Stack Overflow	直接把代码丢给 AI，让它分析
自己测试	让 AI 写测试用例，我来验证
一个人搞定一切	多人分工：架构师 + 代码审查员 + 测试员

1.2 正确的角色定位

在 AI 辅助编程中，你需要扮演三个角色：

- 1. 产品经理（PM）——明确需求、定义边界、验收交付物
- 2. 架构师（Architect）——设计技术选型、模块划分、数据流向
- 3. 代码审查员（Reviewer）——逐行检查 AI 生成的代码，确保正确性和安全性

AI 是你的高级工程师，它干活，你把关。这个分工不能反。

二、提示词工程核心技巧

2.1 万能提示词模板

```
[角色]
你是一位资深 {语言} 开发工程师，精通 {框架/库}。

[背景]
我正在开发一个 {项目描述}，使用 {技术栈}。

[需求]
请实现 {具体功能}，要求：
1. {验收标准1}
2. {验收标准2}
3. {验收标准3}

[约束]
- 不要使用 {不想要的内容}
- 必须遵循 {项目规范}
- 代码需要包含 {注释/类型/文档}

[示例]
参考以下代码风格：
{粘贴示例代码}

[输出]
请输出完整代码，并在代码后附上：
1. 使用说明
2. 已知限制
```

2.2 六大核心技巧

技巧	说明	示例
指定角色	给 AI 一个专业身份	"你是一位安全专家..."
提供上下文	给足够的项目背景	"这是一个 Flutter 项目，使用 Riverpod..."
分步思考	要求 AI 一步步分析	"先分析需求，再设计架构，最后写代码"
给出示例	用示例代码定义期望	"参考以下代码风格：..."
明确约束	告诉 AI 不要做什么	"不要使用任何 any 类型"
要求输出格式	规定输出结构	"输出完整代码 + 使用说明 + 已知限制"

2.3 错误 vs 正确提示词

正确示例: "我需要一个 Flutter 项目的用户登录功能，使用 Firebase Auth。"

要求:

- 1. 支持邮箱 + 密码登录
- 2. 有加载状态和错误提示

- 3. 使用 Riverpod 做状态管理
 - 4. 输入框有基本验证
 - 5. 参考现有项目的代码风格（已附示例）
- 请先列出实现步骤，确认后再输出代码。"

2.4 复杂任务拆解技巧

遇到复杂功能时，不要一次丢一个大需求，拆成多个小步骤：

- 第一步：生成数据模型（User, Order, Item）
- 第二步：生成 API 接口层
- 第三步：生成 UI 组件
- 第四步：集成测试

每步确认后再进入下一步。这样 AI 的输出质量更高，你也更容易审查。

三、开发环境配置最佳实践

3.1 项目级配置 (Rules)

在根目录创建 `.cursorsrules` 或 `.cursor/rules.md`，定义项目规范：

```
# 项目规范
- 语言: Dart 3.5+
- 框架: Flutter 3.24
- 状态管理: Riverpod
- 路由: go_router
- 命名: 文件使用 snake_case, 类使用 PascalCase
- 错误: 统一使用 Result 模式, 返回 {success, data, error}
- 禁止: 使用 any 类型、魔法字符串、硬编码 URL
- 必须: 每个函数有 doc comment、每个 widget 有注释说明用途
```

这样每次对话时，AI 自动遵守项目规范，不需要重复说明。

3.2 AI 模型配置策略

根据任务类型切换不同的 AI 模型：

任务类型	推荐模型	原因
日常代码补全	轻量模型 / Tab 自动补全	速度快，成本低
复杂架构设计	最强模型 (Claude/GPT-4o)	需要深度推理
代码审查	推理能力强的模型	需要发现隐藏问题
调试错误	带代码分析能力的模型	需要理解错误上下文
文档生成	文本生成能力强的模型	需要清晰表达

3.3 Git 集成最佳实践

```
# 每次 AI 生成代码后，养成习惯：
1. 先审查代码（逐行看）
2. 本地运行测试
3. git add + commit 后再进入下一步
4. commit message 注明 AI 参与了哪些修改
```

四、项目级 AI 协作 workflow

4.1 从需求到上线的完整流程

阶段 1：需求分析

提示词：
"我需要一个 [项目描述]，目标用户是 [用户群体]，
核心功能是 [功能1、2、3]。请帮我：
1. 列出用户故事（User Stories）
2. 列出非功能需求（性能、安全等）
3. 列出 MVP 范围（最小可发布版本）
4. 列出后续迭代方向"

阶段 2：架构设计

提示词：
"基于以下需求：[粘贴需求分析]，请帮我设计：
1. 技术选型及理由
2. 模块划分（包含依赖关系图）
3. 数据模型设计
4. API 接口设计
5. 目录结构
请先给出方案，我确认后再细化。"

阶段 3：代码生成（按模块逐个来）

提示词：
"请先实现数据模型层。
技术栈：[列出]
参考项目现有代码风格：[粘贴示例]
输出：完整代码 + 说明"

阶段 4：审查与测试

提示词：
"请作为代码审查员，审查以下代码：
1. 是否存在安全风险？
2. 是否有性能问题？
3. 是否符合项目规范？
4. 是否有更好的写法？
5. 请生成对应的单元测试"

4.2 多 Agent 协作模式

对于大型项目，可以同时启动多个 AI 对话，分工协作：

角色	负责内容	对话方式
架构 Agent	整体设计、技术方案	一次对话定方案
前端 Agent	UI 组件、状态管理	按组件逐个生成
后端 Agent	API、数据库、业务逻辑	按接口逐个生成
测试 Agent	单元测试、集成测试	拿到代码后自动测试



五、代码质量与安全

5.1 AI 生成代码的六大必查项

#	检查项	说明	严重程度
1	API 密钥泄露	硬编码的密钥、Token 必须抽离	高
2	SQL 注入	拼接 SQL 字符串而非使用参数化查询	高
3	XSS 漏洞	直接渲染用户输入未转义	高
4	异常处理缺失	try-catch 不完整，静默吞掉异常	中
5	类型安全问题	使用 any、dynamic 导致运行时	中
6	性能隐患	N+1 查询、未缓存的重复计算	低

5.2 安全最佳实践

5.3 让 AI 做代码审查

"请审查以下代码，重点关注：

- 安全风险（注入、XSS、CSRF 等）
- 边界条件处理（空值、越界、超时）
- 性能问题（循环内查询、不必要的计算）
- 代码风格一致性
- 是否有更优实现方案

请逐条列出问题和建议。"

六、常见陷阱与避坑指南

6.1 新手最常犯的 8 个错误

错误	后果	正确做法
一次给太多需求	AI 输出混乱，部分功能被忽略	拆成小步骤，逐个完成
不审查直接运行	Bug 多，调试成本高	逐行审查 + 单元测试
不更新上下文	AI 用旧代码生成新代码	每次对话都告知当前项目状态
过度依赖一个工具	工具限制 = 你的上限	多工具交叉验证
不写测试	改了一处，坏了三处	AI 写测试，你验证测试
不学基础	无法审查 AI 代码	基础能力是审查能力的底气
复制粘贴不读	引入无用代码、安全风险	每行代码都要理解
不保留版本	改乱了无法回退	频繁 commit，善用 Git

6.2 "AI 幻觉" 应对策略

AI 有时会"一本正经地胡说八道"——引用不存在的 API、编造不存在的库。

应对方法：

- 对 AI 输出的 API/库/版本做验证（查官方文档）
- 让 AI 给出代码后，用 ``dart pub deps`` / ``npm list`` 验证
- 如果编译报错，把错误信息丢给 AI 让它修复
- 重要功能先用最小 demo 验证思路，再集成

七、进阶：Agent 与 MCP

7.1 MCP（模型上下文协议）

MCP 让 AI 能访问外部工具和数据源。相当于给 AI 装上“手和脚”：

能力	说明	示例 MCP
文件系统	读写项目文件	filesystem MCP
数据库	查询/操作数据库	postgres MCP
终端	执行命令行命令	shell MCP
网络	调用 API / 爬取数据	web-fetch MCP
代码分析	静态分析 / 重构	eslint MCP

7.2 Skills（技能定义）

Skills 是用 Markdown 文件定义的“专家模式”：

```
# skills/database/SKILL.md
# 数据库专家
当你处理数据库相关问题时，遵循以下规范：
1. 使用参数化查询防止 SQL 注入
2. 每个查询必须有超时限制（30s）
3. 数据库迁移使用 migration 文件管理
4. 索引设计原则：...
5. 连接池大小 = CPU 核心数 × 2
```

配置后，当你讨论数据库问题时，AI 会自动应用这些规则。

7.3 Hooks（自动化钩子）

让 AI 在特定操作时自动执行任务：

```
{
  "onSave": [
    "npx prettier --write {file}",
    "npx eslint --fix {file}"
  ],
  "beforeCommit": [
    "npm test",
    "git diff --stat"
  ]
}
```

八、工具选型与对比

8.1 2026 主流 AI 编程工具对比

工具	类型	价格	优势	劣势
Cursor	IDE (VS Code fork)	\$0~\$60/月	模型灵活、Composer 多文件编辑	高级功能需付费
Trae	IDE (VS Code fork)	免费	中文最佳、免费、Builder 模式	生态较新
Windsurf	IDE (VS Code fork)	\$0~\$20/月	Flow 模式流程引导、补全速度	团队版调整中
Copilot	IDE 插件	\$10/月	生态最广、IDE 兼容性好	独立能力弱，依赖宿主 IDE
Claude Code	CLI Agent	按量计费	推理最强、代码质量最高	命令行操作，学习曲线
OpenClaw	AI Agent 平台	免费/开源	本地部署、支持自定义模型、MCP 文档	社区较新

8.2 针对 Flutter/Android 开发的推荐组合

--

九、AI 编程 Checklist

每次使用 AI 辅助编程时，按这个清单逐项检查：

写提示词之前

- 1. 我是否已经理解了需求的完整范围？
- 2. 我是否准备好了上下文（技术栈、代码示例、项目规范）？
- 3. 这个需求是否可以拆成更小的步骤？

收到 AI 输出后

- 4. 我逐行审查了代码吗？
- 5. API 调用是否正确？版本是否匹配？
- 6. 有没有安全漏洞（密钥泄露、注入）？
- 7. 有没有多余的、不需要的代码？
- 8. 错误处理是否完整？
- 9. 代码风格是否符合项目规范？

运行和测试后

- 10. 本地编译/运行成功了吗？
- 11. 单元测试通过了吗？
- 12. Edge case（边界条件）测试了吗？
- 13. git commit 了吗？
- 14. 更新了项目文档吗？

